Sreeraaghav Raja
2/18/26
40990068

EE Design 1 Technical Report
Microcontroller ADC & LCD

Introduction

This lab was focused on learning how to use an Analog-to-Digital Convertor (ADC) to properly read analog values from a resistor ladder and print them out to a Liquid Crystal Display (LCD). However, we could not use any communication program to directly connect to the LCD, so we had to emulate our own communication protocol through a unique method. Moreover, this method made the resistance values unreliable, so we had to change the design until the output resistance values met the specifications.

Methods

Background Information:

The Analog-to-Digital Convertor (ADC)

The key to analyzing the ADC is to first understand why we require them. The most fundamental issue with any embedded device is that it struggles to read continuous values. The main challenge with digital devices is that they struggle to discretize continuous signals and the ADC is a fundamental asset in smoothing out this process.

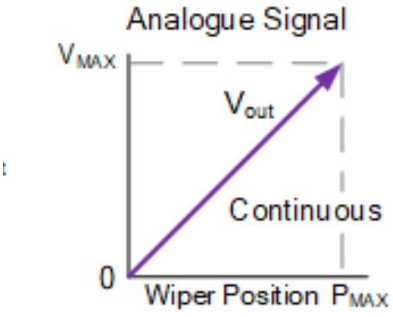

Figure 1: Potentiometer Circuit Vout Reading

Figure 1 is an example of the change in the voltage reading at the output node of a potentiometer [1]. The reading has no breaks in between any of the wiper positions. It is difficult to utilize any number of bits to perfectly match an analog value, so ADCs often have a **resolution** that is based on the number of bits that it can utilize. For instance, a 12-bit ADC will have a resolution of $2^{12} - 1 = 4095$, so the numbers it can read can range from 0 to 4095, unless they are normalized. Moreover, the resolution of the ADC corresponds to the number of steps that an ADC has between its entire range [1].

Figure 2 demonstrates the "same" functionality as the original potentiometer circuit but with a **rotary switch.** A rotary switch operates as complex switch that has multiple position besides

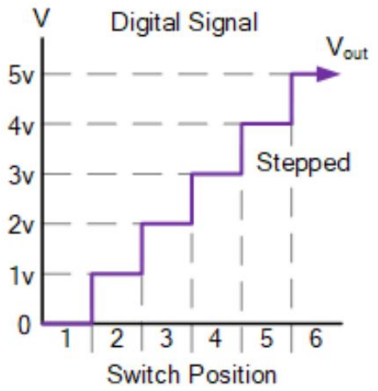

Figure 2: Rotary Switch Circuit Vout Reading

the binary "On" and "Off" positions of a regular switch [2]. Figure 2 differs from Figure 1 based on the different readings for the same range of outputs. The only way to digitally replicate the analog *wiper position* is to have each rotary switch position act as *bin*, a range of values, for a certain amount of wiper positions. Therefore, **Vout** becomes discretized into 6 distinct voltages instead of a complete range of continuous values.

However, the issue of accuracy is trivial if we take the resolution into consideration. Accuracy is equivalent to **span / resolution**. **Span** is the range of voltage values that we are digitizing [3]. Therefore, a smaller span and larger resolution will result in greater accuracy. For instance, a 5V signal with a 5-bit ADC will have a span of 5V, a resolution of 32, and an accuracy of 156.25mV; the calculation is demonstrated by the following equation:

$$Accuracy = \frac{Span}{Resolution} = \frac{5}{32} = 0.15625$$

The Liquid Crystal Display (LCD)

The LCD was the second component that I was unfamiliar with for this lab. It is a device that shows outputs on a display. An LCD works through the geometry and physics of its internal components. At its most fundamental level, an LCD is comprised of a pair of polarizing light filters that and electrodes that sandwich a layer of liquid crystal molecules. The liquid crystals change their orientation if electricity is applied, offering greater control over the display [4].

In this lab, the main way that Pico communicates with the LCD without a backpack is through **Bit-Banging.** Bit-Banging is a method where a device can imitate a communication protocol like UART, I2C, or SPI using pins without using the dedicated hardware for that communication protocol [5]. In this lab, we are imitating a Parallel 4-Bit Protocol with the LCD.py library that I used. However, it is often easier to communicate with LCDs using the Serial-Peripheral Interface Protocol (SPI) because it is faster than both I2C and UART, so the display can refresh faster. However, this was not an option for this lab.

Design Steps:

The most common methodology that I use for most labs that involve hardware is to design the circuit before writing any software. So, I laid out the circuit, mapped Pico's pins to the LCD, and designed the resistor ladder before I wrote any code for the program. Therefore, I'll go over each design decision listed below:

1) **Why a 33kΩ resistor as R1?**
2) **Why 2 Capacitors?**

I decided on the 33kΩ resistor because it was the most effective at keeping the resistance readings close to the expected values even if they were not within the specified error range. Initially, I used a 10kΩ resistor, but that proved to be more error prone in the higher ranges. This is a result of the voltage drop across the 10kΩ test resistor being much smaller than the drop across the larger resistor values when compared to the 33kΩ resistor which would reduce this effect by a factor of 3.3.

The next decision that I made was to add 2 capacitors to reduce noise on both the ADC and the output voltage reading. Capacitors are multi-faceted circuit components that have 2 primary uses: filtering and charge storage. The ADC on the Pico 2 has an internal resistor that draws current thereby reducing the voltage reading on the ADC by approximately 30mV [6]. I used a 0.1µF capacitor at the ADC0 input terminal to offset this internal current. This has the additional effect of reducing any high-frequency noise at that terminal as well. Moreover, I used this same principle in adding the 560pF capacitor at the Vout terminal of the circuit, where we read the output voltage and convert that to a measured resistance value. This capacitor's primary purpose was to filter the noise on the output terminal and, to a lesser extent, reduce the effects that ADC's internal current draw.

Figure 3 shows the basic circuit structure that I decided on for this lab. I added the potentiometer as an additional piece because it controls the contrast on the LCD. Based on the earlier physics explanation of the LCD, the potentiometer effectively controls the angle that the internal liquid crystals are aligned based on the applied voltage at the Vo terminal of the LCD.

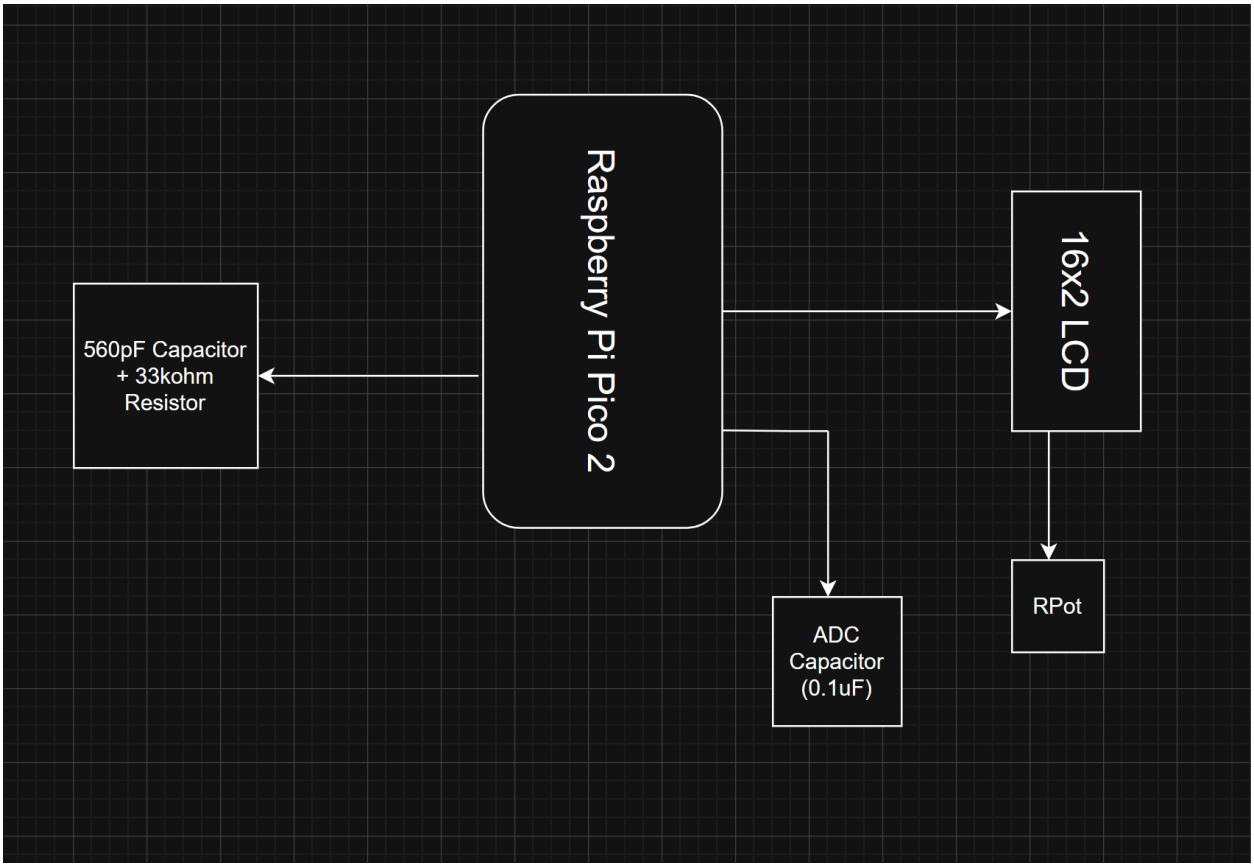

Figure 3: Hardware Block Diagram

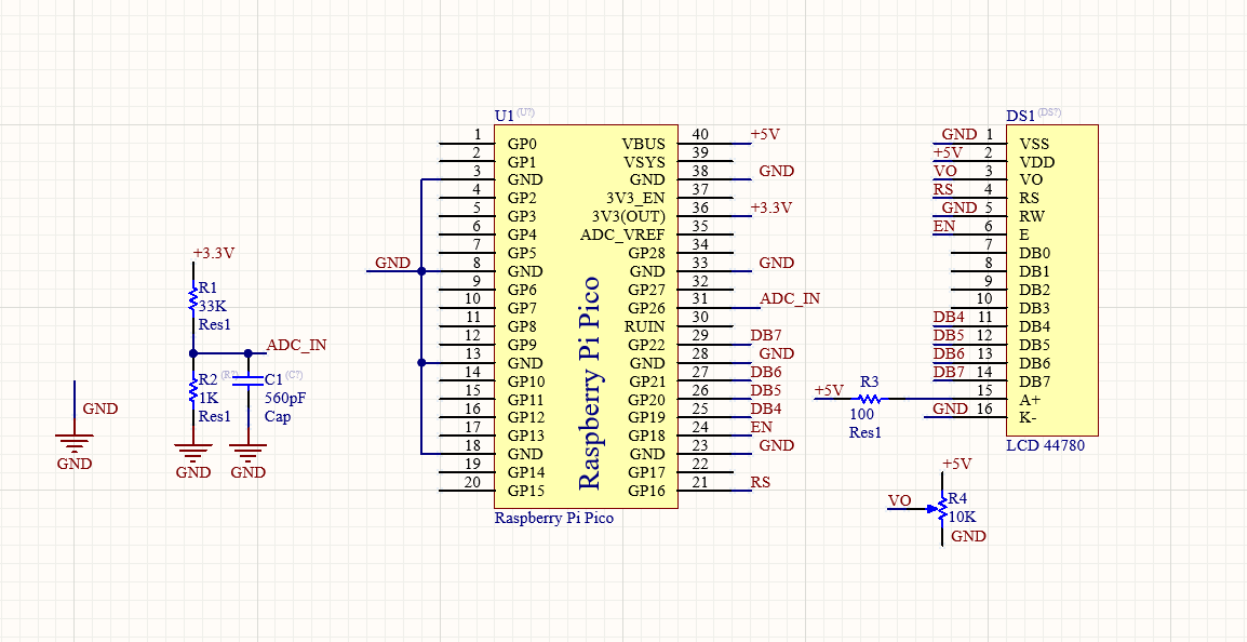

Figure 4: Altium Schematic

The next step of this process was deciding the logic behind the software. While I will not go through every line of my code block, I will go through the most vital ideas. Every embedded program needs to initialize the devices that it uses, so the LCD and ADC were initialized. Then, Pico reads the value from the resistor ladder and converts that to an ADC output. If that value is less than or equal to 0, then it is an error, so the Pico sets the measured resistance value, **r2,** to 10MΩ. Otherwise, **r2** is calculated using the equation below:

$$R2 = \frac{R1 * Vo}{Vref - Vo}$$

1) **R1** is the **test resistance value** which is approximately 33kΩ
2) **Vo** is the **measured voltage value** at the node between R1 and R2
3) **Vref** is **ADC's reference voltage** which is 3.3V

Then, I try to fit the resistance to either a **linear** or **quadratic** equation based on the converted r2 value. I used an experimentally derived threshold for that conditional so that the

data will fit which spec. The effect of this conditional is visible in the table of the expected and measured resistance values.

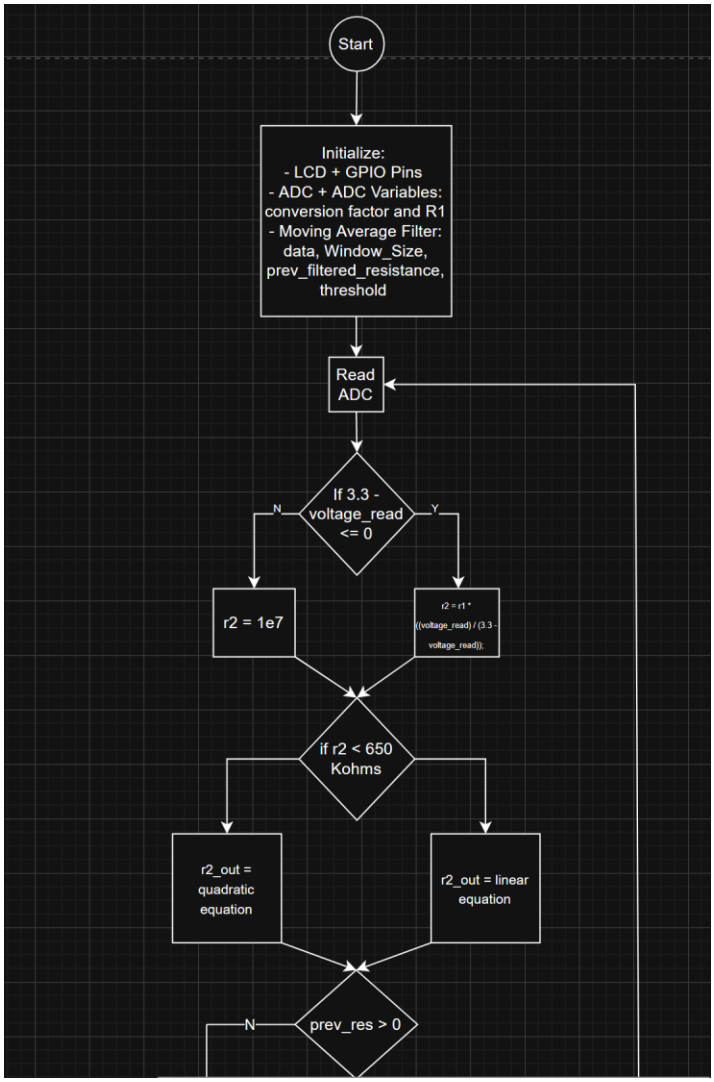

Figure 4: SW Block 1

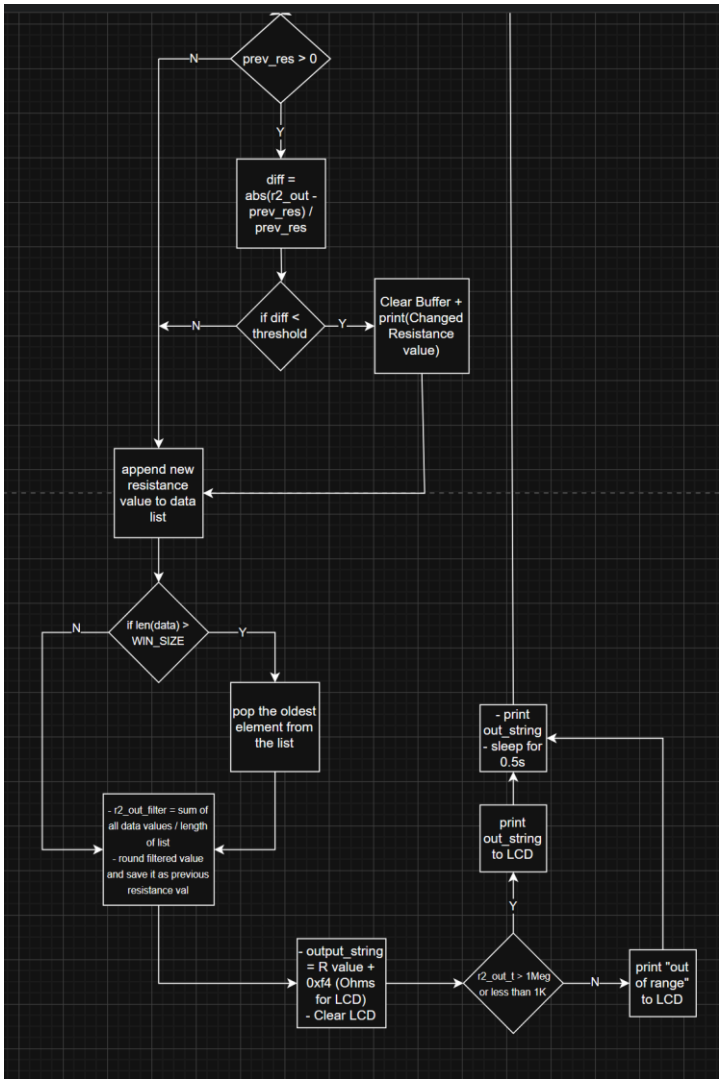

Figure 5: SW Block 2

The largest part of my code dealt with the **rolling average filter** which will be thoroughly explained in the Results section. On a basic level, this section saves the previous resistance value and uses that to determine whether to clear the rolling average buffer. Whether the list is cleared or not, the program appends the most recent resistance value to the data list. If that list is greater than a set size that we are looking at for the filter, then we remove the oldest element from the list. The application of the filter is that we take the average of all the values in the buffer for a specific resistance value and print that to the LCD. The only conditional for that print is that the Pico can only print the resistance value if the value is between 1kΩ and 1MΩ

otherwise it prints “Out of Range” to the LCD. The sleep time is 0.5 seconds to account for the rolling average filter too else the program would be too slow. Finally, the program loops back to reading the ADC value again.

Table 1: Expected vs. Measured Resistance Pre-Correction

Expected Resistance Value (Ω)	***Measured Resistance Value (Ω)***	***Difference***	***Percent Error***
1000	1072.4046	-72.4046	7.24046
5000	5084.26	-84.26	1.6852
10000	10125.574	-125.574	1.25574
15000	15131.353	-131.353	0.875686667
20000	20231.574	-231.574	1.15787
25000	25305.732	-305.732	1.222928
30000	30422.998	-422.998	1.409993333
35000	35573.188	-573.188	1.63768
40000	40575.184	-575.184	1.43796
45000	45628.72	-628.72	1.397155556
50000	50744.524	-744.524	1.489048
55000	55245.784	-245.784	0.44688
60000	61131	-1131	1.885
65000	66246.71	-1246.71	1.918015385
70000	71544.57	-1544.57	2.206528571
75000	76550.32	-1550.32	2.067093333
80000	81663.06	-1663.06	2.078825
85000	86956.14	-1956.14	2.301341176
90000	92295.192	-2295.192	2.550213333
95000	96996.34	-1996.34	2.101410526
100000	103298.768	-3298.768	3.298768
150000	156435.94	-6435.94	4.290626667
200000	211226.82	-11226.82	5.61341
250000	265161.54	-15161.54	6.064616
300000	320565.86	-20565.86	6.855286667
350000	377508.84	-27508.84	7.859668571
400000	435933.33	-35933.33	8.9833325
450000	494532.9	-44532.9	9.8962
500000	567358.7	-67358.7	13.47174
550000	622729.72	-72729.72	13.22358545
600000	681708.436	-81708.436	13.61807267
650000	738883.9	-88883.9	13.67444615
700000	800823	-100823	14.40328571
750000	879696.92	-129696.92	17.29292267
800000	952975.8	-152975.8	19.121975
850000	1030611.04	-180611.04	21.24835765
900000	1092654.4	-192654.4	21.40604444
950000	1173056.8	-223056.8	23.47966316
1000000	1278437.5	-278437.5	27.84375

This table is the expected and measured voltage readings for the circuit I designed before I added the data-fitting equations to the program. I used this table to create data-fitting equations for both a linear and quadratic line. **Figure 6** shows both equations based on **Table 1,** interestingly the quadratic equation first better for all the data values, but it undershoots, demonstrating the effects that the non-idealities have on a circuit.

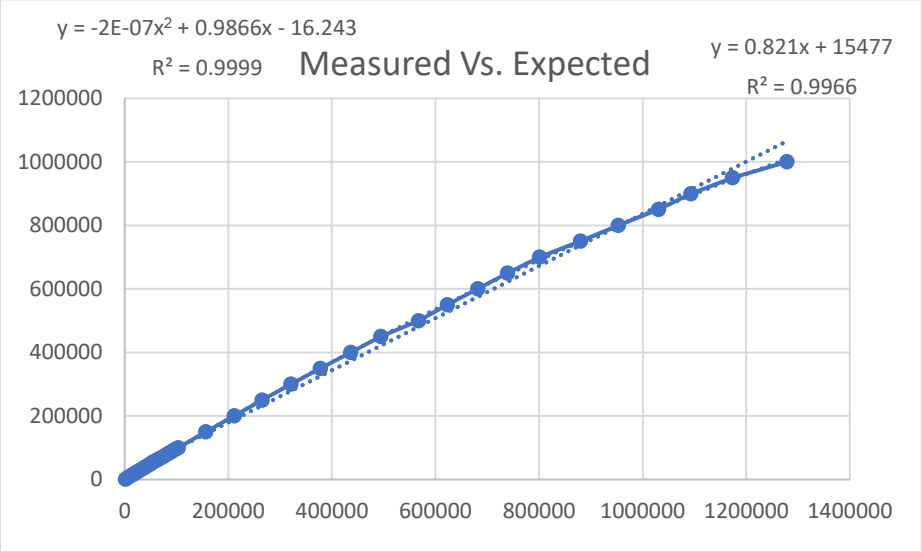

Figure 6: Measured vs. Expected Scatterplot

Results

While it took a few trials, all my results were within the 5% expected range for the resistances. **Table 2** has the same categories as **Table 1** but after I applied the **rolling average filter.** It is important to note that the most inaccurate values are 600kΩ and 650kΩ since that is one of the places where the linear and quadratic fitting equations significantly deviate. Therefore, it was necessary to switch between the linear and quadratic equations.

This is showcased even further with a random resistance value I chose after measuring all the values to demonstrate an accurate measurement. I used a 22kΩ for the LCD to display and it output 21.923924kΩ as the measured value which is within 0.38% of the expected value. However, I mentioned that the **rolling average filter** was important in my design since the

original capacitors were still not giving me values that were nearly this accurate. Therefore, I will explain the theory and code for how I used the rolling average filter below.

Table 2: Expected vs. Measured Resistance Post-Correction

Expected Resistance Value (Ω)	***Measured Resistance Value (Ω)***	***Difference***	***Percent Error***
1000	1037.824	37.824	3.7824
5000	4991.454	-8.546	0.17092
10000	9911.163	-88.837	0.88837
15000	14848.003	-151.997	1.013313333
20000	19858.93	-141.07	0.70535
25000	24833.174	-166.826	0.667304
30000	29727.1	-272.9	0.909666667
35000	34747.964	-252.036	0.720102857
40000	39653.46	-346.54	0.86635
45000	44516.65	-483.35	1.074111111
50000	47567.19	-2432.81	4.86562
55000	54540.296	-459.704	0.835825455
60000	59443.61	-556.39	0.927316667
65000	64487.42	-512.58	0.788584615
70000	69472.38	-527.62	0.753742857
75000	74352.69	-647.31	0.86308
80000	79311.26	-688.74	0.860925
85000	84313.34	-686.66	0.807835294
90000	89192.7	-807.3	0.897
95000	94014.31	-985.69	1.037568421
100000	99648.3	-351.7	0.3517
150000	148860.67	-1139.33	0.759553333
200000	197449.16	-2550.84	1.27542
250000	246860.968	-3139.032	1.2556128
300000	297785.56	-2214.44	0.738146667
350000	344581.04	-5418.96	1.548274286
400000	389496.5	-10503.5	2.625875
450000	438467.96	-11532.04	2.562675556
500000	483009.02	-16990.98	3.398196
550000	533564.3	-16435.7	2.988309091
600000	570338.7	-29661.3	4.94355
650000	624234.2	-25765.8	3.963969231
700000	677162.6	-22837.4	3.262485714
750000	723728.9	-26271.1	3.502813333
800000	787149.22	-12850.78	1.6063475
850000	848560.7	-1439.3	0.169329412
900000	900746.4	746.4	0.082933333
950000	968283.4	18283.4	1.924568421
1000000	1031624.3	31624.3	3.16243

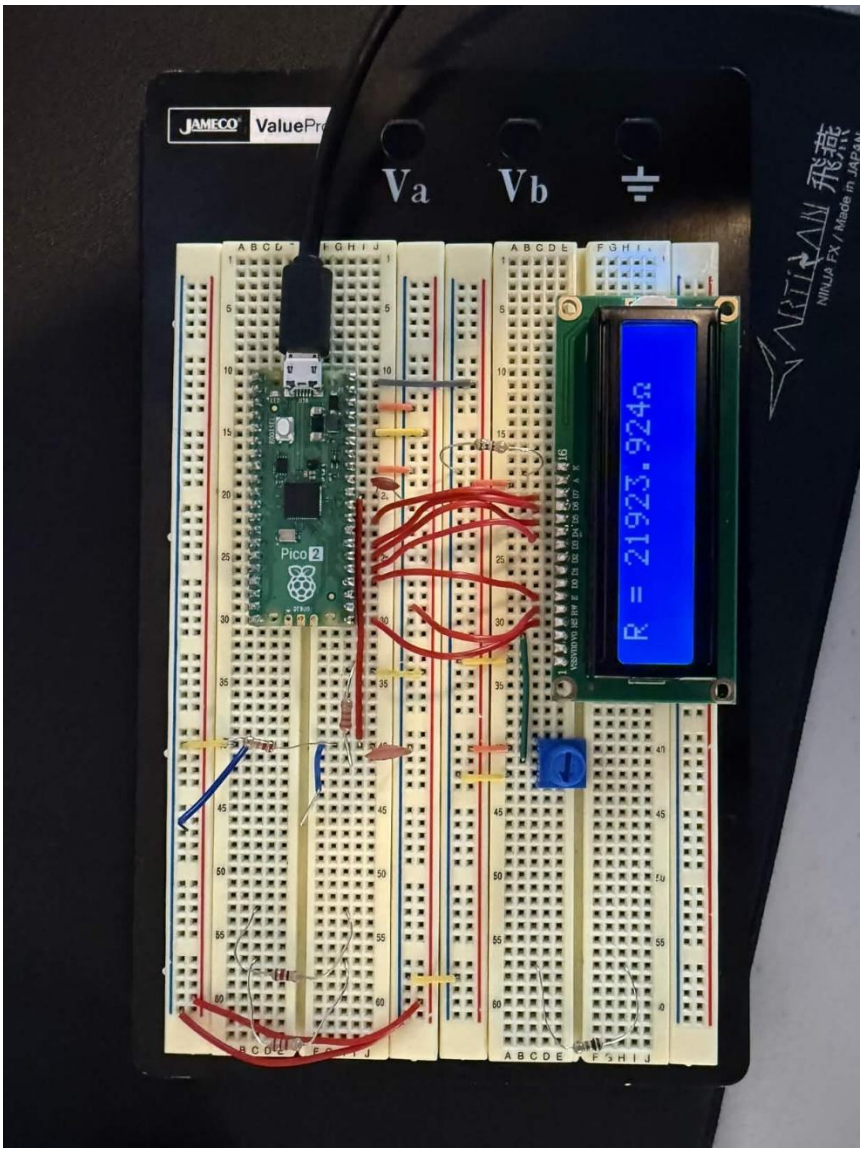

Figure 7: LCD Reading of 220kΩ Resistor

Implementation of the Rolling Average Filter

A rolling average filter is effectively a way to reduce the noise of an input by averaging a certain amount of input values defined by a **Window Size** and outputting that average. My implementation was not ideal since waiting for the ADC to read a Window Size number of values before outputting a single resistance would have a higher time complexity compared to just outputting the values and averaging them as the ADC collected more data. Therefore, the readings would become the most accurate after 25 seconds since my Window Size was set to 50 and the sleep time was 0.5 seconds. The ADC filled the initial data buffer in 25 seconds.

The most difficult part of my rolling average filter measurement was that I had to set a threshold value so that my input buffer would clear and not take in the previous resistance value into account. For example, if I read 100kΩ and then switched to 10kΩ, the LCD would take the 100kΩ values into account until the data buffer is filled with only readings from the 10kΩ resistor ladder. Therefore, by setting a threshold of 0.05, the Pico could effectively clear the list whenever I made large jumps between resistances. The difference had to be 0.05 because of the increments that I was measuring the data with:

1) **Jumps of 5kΩ for readings between 1kΩ and 100kΩ**
2) **Jumps of 50kΩ for readings between 100kΩ and 1MΩ**

If I measured with different increments, I could have made my threshold higher or lower depending on if I increased or decreased the jump size. Setting the logic to switch between resistances was only useful because of how I tested the different measurements: using a **Resistance Decade Box**. If I had manually interchanged resistors for the measurement phase it would have taken significantly longer to measure all the resistances, and I would have taken significantly longer to unravel the issue with the switching logic as.

Overall, this was the primary way that I got the desired resistance values for this lab using software. The hardware issues were uncovered early from reading the datasheet for the ADC, but the extra handling of software was much more difficult to solve. The linearization of the equations was another software improvement that I explained in the methodology, but that was more of a necessity rather than something extra that I had to fix with software.

Conclusion

In this lab, I deepened my understanding of communication protocols with Bit-Banging and internalized the importance of ADCs for microcontrollers as they allow microcontrollers to interact with the environment. This lab solidified concepts that were relatively new in earlier courses like Signals and Systems and Circuits II that are pertinent with this lab. This lab provided a way for me to apply the theory that I gleaned from those classes into an interesting project thereby improving my ability to solve issues as an engineer.

Appendix

```
import machine
import time
import math

from time import sleep
from LCD import LCD

# LCD Instantiation

# GPIO
lcd = LCD(enable_pin=18,           # Enable Pin, int
          reg_select_pin=16,       # Register Select, int
          data_pins=[19, 20, 21, 22]   # Data Pin numbers for the upper nibble.
list[int]
          )

lcd.init()
lcd.clear()


# initialize ADC
voltage_adc = machine.ADC(26)
# 16-bits because micropython upscales the 12 bit value by padding the lower 4
bits with 0
conversion_factor = 3.3 / 65535
# resistor 1 value
r1 = 32978
# linearization equation
lin_eq = 8913 * math.exp(2.57) * 10**(-6)

# Rolling Average Filter
data = []
WIN_SIZE = 50
prev_filtered_resistance = 0
THRESHOLD = 0.05

while True:
    # read the voltage and convert to adc value
    voltage_read = voltage_adc.read_u16() * conversion_factor
    # find r2 value

    if(3.3 - voltage_read <= 0):
        r2 = 1e7
```

```
    else:
        r2 = r1 * ((voltage_read) / (3.3 - voltage_read))

    ## convert the value using linearized equation (used exponential) for normal
values and
    if(r2 < 650e3):
       r2_out = -16.263 + 0.9866 * (r2) - 2e-7 * (r2)**2
    else:
       r2_out = 0.821 * r2 + 15.477e3

    if prev_filtered_resistance > 0:

        diff = abs(r2_out - prev_filtered_resistance) / prev_filtered_resistance

        if diff > THRESHOLD:
            data.clear()
            print("Resistance Changed!")

    data.append(r2_out)

    if len(data) > WIN_SIZE:
        # remove the oldest sample if the length of the array is greater than the
window size
        data.pop(0)

    # take the mean of the data values and the length
    r2_out_filter = sum(data) / len(data)
    r2_out_t = round(r2_out_filter, 3)
    prev_filtered_resistance = r2_out_filter

    # output the adc value to LCD display
    out_string = "R = " + str(r2_out_t) + chr(0xf4)

    # clear before printing
    lcd.clear()

    if (r2_out_t < 1e3 or r2_out_t > 1e6):
        lcd.print("Out of Range")
    else:
        lcd.print(out_string)

    # print out resistance value
    print(out_string)

    sleep(0.5)
```

References

[1] Electronics Tutorials, “Analogue to Digital Converter (ADC) Basics,” *Basic Electronics Tutorials*, Sep. 21, 2020. https://www.electronics-tutorials.ws/combination/analogue-to-digital-converter.html (accessed Feb. 26, 2026).

[2] “A Complete Guide to Rotary Switches,” *uk.rs-online.com*. https://uk.rs-online.com/web/content/discovery/ideas-and-advice/rotary-switches-guide (accessed Feb. 26, 2026).

[3] E. Patrick, “Microcontroller ADC+LCD Lecture,” *Ufl.edu*, 2026. https://mediasite.video.ufl.edu/Mediasite/Play/deec4effed574f24a5a6bd53d55f9b401d (accessed Feb. 26, 2026).

[4] Jeff Tyson, “How LCDs Work,” *HowStuffWorks*, Jul. 17, 2000. https://electronics.howstuffworks.com/lcd.htm (accessed Feb. 26, 2026).

[5] “Bit-Bang | Purdue SIGBots Wiki,” *Purduesigbots.com*, Jul. 05, 2020. https://wiki.purduesigbots.com/electronics/general/bitbang (accessed Feb. 26, 2026).

[6] “Raspberry Pi Pico 2 Datasheet,” Aug. 2024. Accessed: Feb. 26, 2026. [Online]. Available: https://datasheets.raspberrypi.com/pico/pico-2-datasheet.pdf

[7] “Moving Average Filter - an overview | ScienceDirect Topics,” *www.sciencedirect.com*. https://www.sciencedirect.com/topics/engineering/moving-average-filter (accessed Feb. 26, 2026).